Análisis Laboratorio 2 Programación Orientada a Objetos

1. **¿Qué propiedades y métodos tendrá cada clase?**

Clase	Propiedades	Métodos
Pista	codigo, descripcion, tipoEvidencia, nivelImportancia, nivelConfiabilidad.	Constructor, getters, setters y mostrarInformacion().
Ubicacion	codigo, nombre, direccion, nivelRiesgo, estado.	Constructor, getters, setters, mostrarInformacion(), mostrarUbicacionesRegistradas(), mostrarUbicacionesEspecificas(), modificarUbicacion() y descartarUbicacion().
Caso	nombreCaso, codigo, nombreDetective, ubicaciones, pistas.	Constructor, getters, setters, registrarUbicacion(), buscarPosicionUbicacion(), registrarPista(), consultarPista(), buscarPista(), modificarPista(), descartarPista() y mostrarEstadisticas().
Detectives	No declara propiedades; utiliza variables locales en main.	main(), que controla la entrada de datos y el menú.

2. **¿Qué tipo deben tener las propiedades y métodos?**

   - **Pista:** código, descripción y tipo de evidencia son String; importancia y confiabilidad son Integer.
   - **Ubicacion:** código, nombre y dirección son String; riesgo es Integer y estado es Boolean.
   - **Caso:** nombre, código y detective son String; ubicaciones es Ubicacion[] y pistas es List<Pista>.
   - Los getters devuelven el tipo de su propiedad. buscarPista() devuelve Pista y buscarPosicionUbicacion() devuelve int. Los demás métodos son void. Los constructores no declaran tipo de retorno.

3. **¿Cuál propiedad utiliza un arreglo básico y cuál es su tamaño?**

   Ubicaciones almacena objetos Ubicacion. Se crea con new Ubicacion[5]: tiene cinco posiciones, con índices del 0 al 4, inicialmente en null

4. **¿Cuál propiedad utiliza un ArrayList?**

   Pistas está declarada como List<Pista> y se inicializa con new ArrayList<>(). Almacena objetos de Pista y crece conforme se agregan elementos.

5. **¿Cuáles son los modificadores de visibilidad?**

   Todas las propiedades son private; los constructores y métodos son public. Además, main() y los cuatro métodos de Ubicacion que trabajan con el arreglo son static para poder referenciarse entre sí.

6. **¿Qué parámetros requieren los métodos?**

   - **Constructores:** Pista y Ubicacion reciben sus cinco propiedades; Caso recibe nombre del caso, código y nombre del detective.
   - **Setters:** reciben el nuevo valor del tipo correspondiente. Los getters no reciben parámetros.
   - **Ubicaciones:** registrar requiere un objeto Ubicacion y una posición int; mostrar todas requiere Ubicacion[]; consultar una o descartarla requiere el arreglo y la posición; modificar requiere también riesgo Integer y estado Boolean.
   - **Pistas:** registrar recibe Pista; buscar y descartar reciben el código String; modificar recibe código actual, nuevo código, descripción, tipo de evidencia y ambos niveles. setPistas() recibe List<Pista>.
   - buscarPosicionUbicacion() recibe un código String; main() recibe String[] args.
   - Los métodos de información, estadísticas y consultarPista() no reciben parámetros.

7. **¿Cómo se inicializan los objetos y qué se valida?**

   Los constructores asignan los valores mediante setters. Caso también crea el arreglo y la lista vacía.

   Se validan textos nulos o vacíos, riesgo e importancia entre 1 y 10, confiabilidad entre 0 y 100, estado no nulo, posiciones válidas y espacios disponibles. También se rechazan pistas nulas y códigos de pistas duplicados al registrarlas o modificarlas.

   Antes de modificar una ubicación se validan ambos valores; antes de modificar una pista se crea una Pista temporal para validar todos sus datos. Algunos textos usan trim().isEmpty(); los que solo usan isEmpty() todavía aceptan espacios.

8. **¿Cómo se determina si una posición contiene una ubicación o null?**

   Después de comprobar que el arreglo y el índice sean válidos:

```
if (ubicaciones[posicion] != null)
{
   // No esta vacia, tiene una ubicación.
}
else
{
   // La posición está vacía.
}
```

9. **¿Cómo se busca, modifica y elimina dentro del ArrayList?**

   - **Buscar:** buscarPista() recorre la lista y compara códigos con equals(). Devuelve la pista encontrada o null.

   - **Modificar:** modificarPista() busca la pista, verifica que el nuevo código no esté duplicado, valida los datos y aplica los setters.

   - **Eliminar:** descartarPista() busca por código mediante un recorrido por índices y elimina con pistas.remove(i).

10. **¿Qué excepciones pueden producirse y dónde se manejan?**

    - **InputMismatchException:** al ingresar texto donde se espera un entero o un valor distinto de true/false para el estado. Se captura dentro del menú y se limpia la entrada con scan.nextLine().

    - **IllegalArgumentException:** por datos inválidos, valores fuera de rango, códigos duplicados o posiciones ocupadas. Se captura al crear el caso y dentro del menú.

    - **ArrayIndexOutOfBoundsException:** puede ocurrir en registrarUbicacion() si recibe 0, porque valida esa posición, pero escribe en posicion - 1. Actualmente no se captura; debe corregirse la lógica de índices.

    - **NullPointerException:** podría ocurrir al acceder a un objeto nulo; las comprobaciones de null buscan prevenirla.